JobPilot

Integrated Learning + Build Plan

12 Weeks × 22 Hours/Week — 264 Hours Total

(Capacity range: 20–24 hrs/week — plan anchored at 22 hrs)

For: **Pranjal Yadav (Red)**

Starting point: Complete AI/ML zero | 3 YoE Backend/Platform @ Oracle

Version 1.1 – 01 April 2026

Generated: 01 April 2026 – 23:00 IST

MVP ships at Week 7. Full system ships at Week 11. Week 12 = hardening + extended soak testing.

Section 0: The Arithmetic — Why 22 Hours/Week Changes Everything

The PRD estimates 130–160 hours of pure implementation for an engineer already fluent in LangGraph, prompt engineering, RAG, embeddings, and the agent ecosystem. You possess none of these competencies. The minimum viable learning investment before you can write competent agent code is 40–50 hours. That places the realistic total at 170–210 hours.

At 14 hrs/week (v1.0), 12 weeks yielded 168 hours — barely within the lower bound. At 22 hrs/week (v1.1), 12 weeks yields 264 hours: 56 hours of additional capacity deployed as deeper build sessions, richer testing, and built-in recovery buffer for hard weeks. The daily commitment moves from a rigid 2 hrs to approximately 3–3.5 hrs on weekdays and 4–5 hrs on weekends, or any equivalent 22-hour weekly split.

Plan	v1.0 (14 hrs/week)	v1.1 (22 hrs/week)
Weekly hours	14	22 (range: 20–24)
Total hours (12 wks)	168	264
Daily commitment	2 hrs / day, 7 days	~3–3.5 hrs weekdays, 4–5 hrs weekends
Buffer hours vs minimum	~0 hrs	~54–94 hrs buffer
MVP milestone	End of Week 8	End of Week 7 (accelerated)
Full system	End of Week 12	End of Week 11; Week 12 = hardening

The Two-Stage Contract (Revised)

Stage 1 (Weeks 1–7): MVP — Scout, Resume, Application, Tracker, Email Intelligence, Outreach agents. Functional end-to-end pipeline. You can begin applying to jobs. 154 hours consumed.

Stage 2 (Weeks 8–11): Full System — Adaptive Resume Engine (4-agent tournament + Mediator), Follow-up Agent, Prometheus + Grafana dashboards, MCP servers, Docker Compose deployment, integration testing. 88 hours consumed.

Stage 3 (Week 12): Hardening — Extended soak testing, documentation polish, load testing at scale, VPS deployment. 22 hours. Total: 264 hours.

Section 1: What Transfers From Your Backend Experience

Your Oracle experience provides substantial scaffolding. The following mappings accelerate specific learning modules:

Your Existing Skill	AI/ML Concept	How It Maps
REST API design, Dropwizard	LLM API integration	LLM APIs are HTTP endpoints with JSON payloads. You already know how to call them.

Your Existing Skill	AI/ML Concept	How It Maps
Kubernetes orchestration, OKE	LangGraph state machines	Graph-based state transitions with retry logic. Conceptually similar to K8s controller reconciliation loops.
CI/CD pipelines, TeamCity	Agent pipelines + APScheduler	Sequential and parallel task execution with failure handling. Direct analogue.
Control Plane / Data Plane	Agent architecture	Agents are autonomous services with defined I/O contracts. Your CP/DP mental model applies.
Docker, Helm, Terraform	Docker Compose deployment	Trivial. You already know containerisation.
Prometheus (OCI monitoring)	Prometheus + Grafana	Same stack. Metrics exposition, scrape configs, alerting rules — all familiar.
PostgreSQL (PAIOS)	pgvector, SQLAlchemy	pgvector is an extension on the DB you already run. New concept is vector similarity, not the database.
Security vuln remediation	OAuth 2.0 (Gmail API)	Token management, scope restrictions, credential rotation — security primitives you already practise.

What Has No Backend Equivalent

Prompt engineering (structured system prompts, few-shot examples, output format enforcement), RAG pipelines (embedding generation, vector similarity search, context window management), multi-agent deliberation (tournament scoring, consensus mechanisms), and LLM-as-judge evaluation patterns. These require deliberate learning — no amount of backend experience shortcuts them.

Section 2: Core Learning Resources

Each resource is cited at the week it is consumed. This section serves as the master reference list.

Domain	Resource	Type	When Used
LLM Fundamentals	Andrej Karpathy — "Intro to LLMs" (YouTube, 1hr)	Video	Week 1, Day 1
LLM Fundamentals	OpenAI Tokenizer (tiktoken playground)	Tool	Week 1, Day 2
Prompt Engineering	Anthropic Prompt Engineering docs (docs.anthropic.com)	Docs	Week 1, Days 3–4
Prompt Engineering	OpenAI Prompt Engineering Guide	Docs	Week 1, Day 5
Python for AI	LangChain official docs — quickstart	Docs	Week 2, Day 1
LLM APIs	DeepSeek API docs, Qwen (DashScope) API docs	Docs	Week 2, Days 2–3
Agent Frameworks	LangGraph official tutorial (langchain-ai.github.io)	Docs	Week 4, Days 1–3

Domain	Resource	Type	When Used
Agent Frameworks	Harrison Chase — LangGraph walkthrough (YouTube)	Video	Week 4, Day 1
RAG & Vectors	pgvector docs + LangChain RAG tutorial	Docs	Week 3, Days 1–4
RAG & Vectors	Pinecone Learning Center — "What are Embeddings?"	Article	Week 3, Day 1
Gmail API	Google Gmail API Python quickstart	Docs	Week 5, Day 1
MCP Protocol	Anthropic MCP specification (modelcontextprotocol.io)	Docs	Week 9, Day 4
Evaluation	LangSmith / LangFuse docs (observability)	Docs	Week 10, Day 6
Playwright	Playwright Python docs — async API	Docs	Week 5, Day 3
LaTeX	pdflatex CLI reference, Overleaf guides	Docs	Week 3, Days 5–7

Section 3: Week-by-Week Plan

Every row specifies what you learn and what you build on that day. At 22 hrs/week across 7 days, each day averages 3.1 hours. Allocation is flexible: learn-heavy days run 40–60 min learn + balance build; build-heavy days run 15–20 min review + balance build. The expanded hours create genuine buffer within each week rather than mandating a rigid split.

STAGE 1: MVP (Weeks 1–7) — Functional Job Application Pipeline
Scout → Resume → Application → Tracker → Email Intelligence → Outreach.
End-to-end pipeline with Telegram HITL. You can begin applying to real jobs at the end of Week 7.
154 hours consumed at Stage 1 completion.

Week 1: LLM Fundamentals + Project Scaffolding

Phase: *Foundation* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**LLM Concepts**	Watch Karpathy "Intro to LLMs." Understand tokens, context windows, temperature, top-p. Extended time: experiment in OpenAI Playground.	Set up project repo. docker-compose.yml with PostgreSQL. FastAPI /health endpoint. Explore 3 different prompts manually against DeepSeek.	**Repo initialised + 3 prompt experiments documented**
2	**Tokenisation**	tiktoken playground: paste your resume, observe token count. Understand cost	Install all Python deps (langchain, openai, httpx, etc.). Create config.yaml	**Config system + token counter**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
		implications at scale. Read pricing pages for DeepSeek, Qwen, Claude.	parser and .env. Write token-counting utility.	
3	**Prompt Eng I**	Anthropic docs: system prompts, role prompts, output format enforcement, XML tag patterns. Take notes on what breaks structured output.	Write JD scoring system prompt (PRD §5.2). Test against 5 real JDs via DeepSeek API. Iterate on format until JSON is reliable.	**Scoring prompt v1 — tested on 5 JDs**
4	**Prompt Eng II**	Few-shot prompting, chain-of-thought. Structured JSON output enforcement. Read OpenAI prompt engineering guide.	Write JD keyword extraction prompt (PRD §6.2). Test extraction against 5 real JDs. Compare few-shot vs zero-shot quality.	**Extraction prompt v1 — comparison logged**
5	**Cascading LLM**	DeepSeek and Qwen API docs. Rate limits, error codes, retry semantics. Understand fallback rationale.	Implement CascadingLLM class (PRD §4.2). 5-tier chain with Ollama terminal. Test each tier manually.	**Cascade client — all 5 tiers verified**
6	**Telegram Bot**	python-telegram-bot v20: handlers, inline keyboards, callback queries. Study existing bot examples.	Create Telegram bot. Implement send_job_notification() with Apply/Skip/Details buttons. Test all three flows.	**Telegram bot live — all buttons functional**
7	**Integration + Buffer**	Review and refactor all Week 1 code. Code review checklist: naming, error handling, logging.	Wire CascadingLLM + scoring prompt + Telegram. Manual test: paste JD → scored notification. Fix any issues. Write Week 1 test cases.	**End-to-end test passing + test_week1.py**

Week 2: Scout Agent — Job Discovery

Phase: *Scout Agent Build* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Web Scraping**	httpx async basics. BeautifulSoup DOM parsing. Rate limiting patterns. Study robots.txt conventions.	Lever generic scraper: {company}.lever.co/api/postings JSON. Parse title, location, description. Test against 5 companies.	**Lever scraper — 5 companies confirmed**
2	**ATS Scrapers**	Greenhouse API patterns. Ashby page structure. feedparser for RSS/Atom. Compare response formats.	Greenhouse scraper. RSS auto-detection probe. feedparser integration. Verify data normalisation.	**3 ATS scrapers — unified schema**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
3	**Title Matching**	Fuzzy string matching: Levenshtein distance, token overlap. rapidfuzz library. Test sensitivity thresholds.	Title pre-filter (PRD §5.2 Stage 1). Hardcoded title list + fuzzy matching. Tune threshold on 20 sample JDs.	**Title filter — threshold calibrated**
4	**Deduplication**	SHA-256 hashing for dedup. PostgreSQL index for O(1) lookup. Read about idempotent pipelines.	Dedup pipeline (PRD §5.2 Stage 4). PostgreSQL seen-postings table. Geographic filter. Test with duplicate injection.	**Dedup + geo filter — zero duplicates**
5	**Full Pipeline**	Pipeline assembly: scrape → title filter → LLM score → dedup → threshold → notify. Study APScheduler docs.	Wire full Scout Agent. APScheduler: 10min tier-1, 4hr tier-2. Run against live boards for 1 hour.	**Scout Agent v1 — running live**
6	**LinkedIn/ Naukri**	LinkedIn Jobs scraping constraints. Naukri API/scraping patterns. Rate limit research.	LinkedIn jobs scraper (httpx). Naukri scraper. Integrate into Scout. Test all sources.	**All sources live — unified feed**
7	**Testing + Buffer**	Edge cases: empty JDs, non-English postings, malformed HTML, ATS downtime.	test_scout.py. Run against 10 real career pages. Fix all failures. Write integration test for dedup idempotency.	**Scout fully tested — 10/10 pages pass**

Week 3: RAG + Resume Agent

Phase: *RAG & Vectors + Resume Build* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Embeddings**	What are embeddings. Cosine similarity. Pinecone Learning Center article. Experiment with sentence-transformers interactively.	Install pgvector on PostgreSQL. Create embeddings table. Test with sentence-transformers on 10 JD samples.	**pgvector running — 10 embeddings stored**
2	**RAG Pipeline**	LangChain RAG tutorial: loading, splitting, embedding, retrieval. Compare chunk sizes.	Embed your resume + 10 real JDs. Query by cosine similarity. Evaluate top-k accuracy manually.	**RAG prototype — relevance verified**
3	**RAG in Scout**	Hybrid scoring: LLM + embedding similarity for relevance. Read on late vs early fusion.	Add pgvector pre-scoring to Scout Agent. Compare LLM-only vs hybrid on 20 JDs. Log precision/recall.	**Hybrid scoring — comparison logged**
4	**LaTeX Parsing**	Your .tex template: \\section,	LaTeX parser: sections, bullets, skills into structured	**LaTeX parser — all sections extracted**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
		\\resumeSubheading, \\resumeItem. Regex extraction patterns.	intermediate representation. Test on your own resume.	
5	**Bullet Rewriting**	Resume rewriting prompt constraints (PRD §6.2 Step 3). No-fabrication rules. Read Harvard action verb list.	Bullet rewriting via CascadingLLM. Harvard action verb injection. Test on 5 JD pairings. A/B compare output.	**Rewriter v1 — 5 JD pairings tested**
6	**LaTeX Compile**	pdflatex CLI. Error parsing. Auto-fix for unescaped &, %, $.	texlive in Docker. compile_latex() function (PRD §6.2 Step 6). Test compilation with 3 resume variants.	**Compiler working — 3 PDFs generated**
7	**ATS Scoring + Buffer**	Keyword match rate. Bullet reordering. Skills section logic.	ATS score estimation. Reordering logic. Resume Agent end-to-end test on 5 JD pairings. Fix edge cases.	**Resume Agent v1 — 5/5 pairings pass**

Week 4: LangGraph + Tracker + Application Agent

Phase: *Agent Orchestration* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**LangGraph I**	LangGraph tutorial: StateGraph, nodes, edges, conditional routing. Harrison Chase video. Take notes on state schema design.	Minimal LangGraph: 2 nodes (Scout → Resume) with shared state. Test state persistence between nodes.	**First graph runs — state schema defined**
2	**LangGraph II**	HITL checkpoints, interrupt_before, human approval nodes. State persistence.	Add Telegram HITL checkpoint between Scout and Resume nodes. Test approve/reject flow.	**HITL checkpoint — both flows verified**
3	**LangGraph III**	Parallel execution, fan-out/fan-in. Error handling and retry in graphs.	Expand graph: Scout → [HITL] → Resume → [HITL] → Application (stub). Full state schema.	**3-node graph — full state schema**
4	**Tracker Agent**	openpyxl: workbooks, sheets, conditional formatting, column sizing.	Tracker Agent (PRD §9). 30-column schema. Auto-populate from Scout + Resume state. Test with 10 synthetic jobs.	**Tracker v1 — 10 rows auto-populated**
5	**Tracker Polish**	Conditional formatting rules: red/yellow/green rows. Freeze panes. Auto-column width.	Conditional formatting (PRD §9.3). Daily summary Telegram message at 21:00. Test formatting rules.	**Tracker polished — formatting verified**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
6	**Application Agent**	Lever POST /postings/{id}/apply. Gmail gmail.send for email applications. Study rate limits.	Application Agent (PRD §7). Lever API + email + manual fallback. Test against Lever sandbox.	**Application Agent — Lever + email working**
7	**Integration + Buffer**	End-to-end integration patterns. Retry on partial failure.	Run pipeline against 5 real postings. Fix integration bugs. Write test_pipeline.py for 4-agent chain.	**4-agent pipeline — 5/5 postings pass**

Week 5: Gmail OAuth + Outreach Agent

Phase: *Application Completion* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Gmail OAuth**	Google Cloud Console: project, Gmail API, OAuth credentials. OAuth 2.0 flow in depth.	setup_gmail_oauth.py. Obtain refresh token. Test gmail.readonly scope. Store credentials securely.	**Gmail auth done — token stored securely**
2	**Gmail Send**	Gmail API: messages.send(), MIME construction, attachments. Test in sandbox.	Email application submission via Gmail API. Resume PDF attachment. Test with own inbox.	**Email submission — attachment verified**
3	**Playwright**	Playwright Python async: browser launch, selectors, form filling. Study anti-bot patterns.	Playwright install. LinkedIn Easy Apply prototype. Test with real postings. Handle rate limits.	**Playwright basics — 2 form fills working**
4	**Form Filling**	applicant_profile.yaml auto-population. ATS form field mapping across platforms.	Form auto-population (PRD §7.3). Test against Lever and Greenhouse forms. Handle field variations.	**Auto-fill working — Lever + Greenhouse**
5	**Confirmation Gate**	Confirmation gate flow (PRD §7.2). UX design for Telegram approval.	Telegram Confirm/Cancel/View Resume buttons. Wire into LangGraph. Test full approval flow.	**Confirmation gate — all 3 buttons tested**
6	**Outreach Prep**	Apollo.io API: search_people, free tier (50/month). Hunter.io API. Compare data quality.	Apollo.io account. find_recruiters_apollo() (PRD §8.1). Test with 5 companies. Log data quality.	**Apollo integration — 5 companies tested**
7	**Outreach Agent + Buffer**	Dual-variant messages (PRD §8.2). Rate limits (PRD §8.3). LinkedIn messaging constraints.	Outreach Agent: 2 variants per recruiter. Telegram selection UI. Test end-to-end with 3 real recruiters.	**Outreach Agent v1 — 3 live tests done**

Week 6: Email Intelligence Agent

Phase: *Email Classification* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Gmail Sync**	users.history().list(), historyId incremental sync. Label management. Study Gmail quotas.	Incremental email sync (PRD §10.1). Store historyId in PostgreSQL. Test sync with real inbox.	**Email sync — incremental confirmed**
2	**Pre-filters**	Domain matching vs tracker. Regex keyword classification (PRD §10.2). False positive analysis.	Stage 1 (domain filter) and Stage 2 (keyword pre-filter). Test against real inbox. Log FP/FN rates.	**Pre-filters done — FP/FN logged**
3	**LLM Classify**	8-category taxonomy (PRD §10.2). Confidence thresholds. Few-shot examples for each category.	Classification prompt. Stage 3 LLM classification via CascadingLLM. Test on 20 real emails.	**Classifier done — 20/20 manual verified**
4	**Tracker Update**	Auto-updating tracker from classifications. Status transitions. Conflict resolution.	Email Intelligence → Tracker Agent. Auto-label Gmail (PRD §10.4). Test status transitions.	**Auto-updates — 5 status flows tested**
5	**Feedback Store**	Outcome weight system (PRD §10.3). Adaptive Resume Engine data schema.	Feedback data collection. Per-application outcome weights in PostgreSQL. Schema migration.	**Feedback store — schema live**
6	**Privacy + Hardenin g**	Privacy constraints (PRD §10.5). Classification-only persistence. Token encryption approach.	Privacy audit. No raw email storage. Token encryption. Run 50-email batch, verify no raw data persisted.	**Privacy audit passed — 50-email batch**
7	**Full 6-Agent Pipeline**	Integration patterns across 6 agents. Shared state schema review.	End-to-end test with 10 real postings across all 6 agents. Fix integration issues. Write test_6agents.py.	**6-agent pipeline — 10/10 postings pass**

Week 7: Pipeline Hardening + MVP Ship

Phase: *Stabilisation + MVP Deployment* | **Hours: 22 hrs**

The extra hours vs v1.0 allow MVP hardening and deployment to be merged into Week 7 rather than split across Weeks 7–8.

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Error Handling**	Retry strategies. Graceful degradation. Dead letter queues.	Structured error handling for all 6 agents. Exponential backoff. Test each failure mode.	**Error handling — all failure modes covered**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
2	**Rate Limits + Scheduling**	Per-source limits. APScheduler cron, job persistence, timezone (Asia/Kolkata).	Rate limiter middleware. Token bucket per service. All APScheduler jobs: Scout 10min/4hr, Email Intel 5min, summary 21:00.	**Rate limiters + scheduler — all jobs live**
3	**State Recovery + Logging**	LangGraph checkpointing. PostgreSQL-backed persistence. Structured logging.	Checkpoint persistence. Test crash recovery. Logging across all agents. Correlation IDs.	**State recovery + logging done**
4	**Dockerfile + Compose**	Multi-stage builds. Python + TeX Live. docker-compose: services, volumes, networking, healthchecks.	Dockerfile with pdflatex, Python deps, Playwright browsers. docker-compose.yml: app + PostgreSQL. Shared resume volume.	**Docker Compose running**
5	**Data Seeding + DB**	Schema management. Initial data (companies.yaml, Harvard verbs).	init_database.sql. Seed 200 companies with ATS annotations. Verify all migrations.	**DB seeded — 200 companies**
6	**Live Trial**	Run against real job boards for a full day. Monitor logs.	Apply to 5–10 jobs with Telegram approval. Verify tracker accuracy. Fix critical issues.	**LIVE DEPLOYMENT — first real applications**
7	**Retrospective + Docs**	README: quickstart, config, architecture, troubleshooting. Architecture diagram.	README.md. Document all env vars. Architecture diagram. Fix any critical issues from Day 6.	**MVP SHIPPED — fully documented**

✅ MVP Milestone (End of Week 7) — 154 Hours Consumed

Pipeline is live. Scout discovers jobs, Resume Agent tailors LaTeX, Application Agent submits with approval, Tracker auto-populates, Email Intelligence classifies, Outreach contacts recruiters. You are actively applying to jobs.

Achieved one week earlier than v1.0 schedule due to 22 hrs/week capacity.

STAGE 2: Full System (Weeks 8–11) — Intelligence + Observability

Adaptive Resume Engine, Follow-up Agent, Prometheus + Grafana, MCP servers. The system becomes self-improving and fully observable.

88 hours in this stage (vs 56 in v1.0), enabling deeper testing and richer implementation of each component.

Week 8: Adaptive Resume Engine

Phase: *Multi-Agent Tournament* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Feedback Data**	Positive/negative signal queries (PRD §12.2). Outcome weights.	SQL feedback collection queries. Feedback aggregation module. Test with synthetic data.	**Feedback queries — aggregation verified**
2	**Tournament I**	4-agent architecture (PRD §12.3). Parallel agents. Role specialisation.	Keyword Agent and Structure Agent. Base resume + feedback + system prompt.	**2 tournament agents — outputs compared**
3	**Tournament II**	Narrative Agent (phrasing). Market Agent (emerging JD trends).	Narrative and Market Agents. Wire all 4 for parallel execution. Compare 4 outputs on same JD.	**4 tournament agents — parallel running**
4	**Judge + Score**	LLM-as-judge. 5-dimension rubric (PRD §12.4). Truthfulness 2x weight.	Judge Agent. Score proposals. Determine winner and margin. Test on 5 JDs.	**Judge scoring — 5 JDs evaluated**
5	**Deliberation**	Mediator Agent: triggered when margin < 5. Merge synthesis (PRD §12.4).	Mediator Agent. Re-score merged version. 3-candidate selection. Test deliberation trigger.	**Deliberation done — trigger verified**
6	**Versioning + Safety**	Git-like history (PRD §12.5). No-fabrication, performance floor, /revert, /pin.	version_manager.py. resume_versions/ directory. Symlinks. changelog.md. All guardrails.	**Version control + guardrails — /revert tested**
7	**ARE Integration + Buffer**	Full pipeline integration. APScheduler timing.	APScheduler 06:00/18:00. Cold start logic. End-to-end ARE test on 5 JDs. Fix issues.	**ARE complete — 5/5 JDs pass**

Week 9: Follow-up Agent + MCP Servers

Phase: *Follow-up + MCP* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Follow-up Trigger**	Trigger logic (PRD §11.1). Day 7 + Day 14 SQL. Skip rules.	Follow-up candidate query. APScheduler daily 10:00 AM IST. Test with synthetic applications.	**Follow-up trigger — scheduler running**
2	**Email Draft**	Follow-up prompt (PRD §11.2). Prohibited phrases. Plain text constraints.	LLM follow-up drafting. Day 7 (4 sentences) and Day 14 (2, final). Test on 5 applications.	**Draft generation — 5 drafts reviewed**
3	**Approval Flow**	Telegram UI (PRD §11.3). Send/Edit/Skip/Delay buttons.	Telegram follow-up approval. Delay 3 days logic. Gmail API send. Test all 4 button flows.	**Follow-up Agent — all flows verified**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
4	**MCP Basics**	MCP spec: servers, tools, resources. Streamable HTTP. FastMCP.	jobpilot-scout MCP server (PRD §14.1). search_jobs(), get_job_details(). Test with Claude desktop.	**Scout MCP — Claude desktop connected**
5	**MCP Resume**	MCP tool schemas. Input/output contracts. Error handling.	jobpilot-resume MCP server (PRD §14.2). tailor_resume(), estimate_ats_score(). Test both tools.	**Resume MCP — both tools verified**
6	**MCP Outreach**	MCP testing patterns. Client integration.	jobpilot-outreach MCP server (PRD §14.3). find_recruiters(), draft_outreach(). Test with Claude.	**3 MCP servers — all tools functional**
7	**9-Agent Graph + Buffer**	Full LangGraph with 9 nodes. State schema completeness.	Update LangGraph with Follow-up + ARE nodes. Full integration test with 5 real postings. Fix issues.	**9-agent graph — 5/5 integration tests pass**

Week 10: Prometheus + Grafana + Observability

Phase: *Monitoring* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Metrics Def**	prometheus_client: Counter, Gauge, Histogram. /metrics endpoint design.	Define 15+ metrics (PRD §13.2). Expose /metrics via FastAPI. Verify all metrics emit.	**Metrics endpoint — 15+ metrics live**
2	**Prometheus**	prometheus.yml scrape config. alert_rules.yml. Scrape interval trade-offs.	Prometheus in docker-compose. 15s scrape. 4 alert rules. Verify scrape in Prometheus UI.	**Prometheus live — 4 alert rules active**
3	**Grafana I**	Provisioning: datasources, dashboard JSON. Panel types.	Grafana in docker-compose. Prometheus datasource. Funnel dashboard. Verify all panels.	**Funnel dashboard — all panels rendering**
4	**Grafana II**	PromQL: conversion rates, timing distributions, cost tracking.	4 remaining dashboards: daily activity, resume evolution, LLM cost, outreach. Test all queries.	**5 dashboards — all PromQL verified**
5	**Alerting**	Grafana alerting: channels, conditions, silence rules.	Telegram as alert channel. Test all 4 rules by injecting failures. Verify Telegram receipt.	**Alerts working — 4 rules tested live**
6	**LangSmith Tracing**	LangSmith/LangFuse: LLM tracing, token usage, latency.	LangSmith tracing in CascadingLLM. Tag by agent and task. Run 20-JD batch, review traces.	**Tracing live — 20 traces reviewed**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
7	**Eval Suite + Buffer**	LLM-as-judge for quality validation (quality.py). Evaluation rubric design.	quality.py: resume rewriting quality scoring. Threshold retry. Run on 10 resumes. Fix issues.	**Eval pipeline — 10/10 resumes scored**

Week 11: Final Deployment + Comprehensive Testing

Phase: *Full System Deployment + Test Suite* | **Hours: 22 hrs**

In v1.0 this was Week 12 (14 hrs). The additional 8 hours enable a more thorough test suite and a 24-hour soak test within the same week.

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Docker Polish**	Health checks, restart policies, volumes, log rotation.	Final docker-compose: app + PostgreSQL + Prometheus + Grafana + Ollama. Verify all healthchecks.	**Final compose — all services healthy**
2	**Testing I**	Integration test design: agent pairs, pipeline, error paths.	test_resume.py, test_cascade_llm.py, test_email_intelligence.py. Achieve >90% coverage.	**Test suite I — 3 files, >90% coverage**
3	**Testing II**	ARE testing: tournament, deliberation, revert.	test_adaptive_resume.py, test_outreach.py, test_follow_up.py, test_metrics.py. All passing.	**Test suite II — 4 files, all passing**
4	**Load Test**	50 concurrent postings. CascadingLLM under rate limit. Failure injection.	Load test: 50 synthetic postings. Verify no data loss, correct dedup. Fix any bottlenecks.	**Load tested — 50 concurrent, 0 data loss**
5	**Docs + VPS (Optional)**	Hetzner CX22 setup. SSH, Docker, firewall. README completeness.	Update README. Troubleshooting guide. Optional VPS deploy. Verify remote docker-compose up.	**Docs complete — VPS optional done**
6	**24hr Soak Test**	24-hour unattended run. Prometheus dashboards. Anomaly patterns.	24hr soak test. Review metrics. Fix anomalies. Verify ARE fires at scheduled intervals.	**Soak test passed — 24hr unattended**
7	**Ship + Retrospective**	Final PRD requirements review. Gap analysis. Changelog.	Tag v1.0 release. Update changelog. Final retrospective notes.	**JOBPILOT v1.0 SHIPPED**

✅ Full System (End of Week 11) — 242 Hours Consumed

All 9 agents. Adaptive Resume Engine with 12hr cycles. Prometheus + Grafana (5 dashboards). 3 MCP servers. Follow-up Agent (day 7 + 14). Docker Compose deployment. Full test suite. 24hr soak test passed.

Achieved one week earlier than v1.0 schedule.

STAGE 3: Hardening Week (Week 12) — Exclusively Available in v1.1
22 additional hours that do not exist in v1.0. Deploy to production VPS. Stress test at 3x load. Polish documentation. Address all remaining technical debt.
Optional: begin portfolio case study writeup for job applications.

Week 12: Hardening, Scale Testing + Portfolio

Phase: *Production Hardening* | **Hours: 22 hrs**

Day	Topic	Learn (Concept)	Build (Apply)	Deliverable
1	**Production VPS**	Hetzner/DigitalOcean production setup. Domain, SSL, reverse proxy.	Production VPS deploy. Nginx reverse proxy. SSL certificate. Verify all agents run remotely.	**Production server live**
2	**3x Load Test**	Stress testing methodology. Bottleneck identification.	3x load: 150 concurrent postings. Profile slow queries. Add DB indexes where needed.	**3x load — bottlenecks resolved**
3	**Security Hardening**	Secrets management. Network isolation. Least privilege review.	Rotate all credentials. Docker network isolation. Review all permission scopes. Penetration checklist.	**Security audit complete**
4	**Monitoring Tuning**	Alert fatigue. Threshold calibration. Dashboard readability.	Tune alert thresholds from Week 11 soak data. Add missing metrics. Polish dashboard layout.	**Dashboards production-ready**
5	**Technical Debt**	Code review: naming, complexity, duplication. Refactoring patterns.	Address all TODO/FIXME comments. Refactor any components flagged in retrospective.	**Tech debt cleared**
6	**Portfolio Case Study**	How to write a technical case study for SDE interviews.	Write 1000-word JobPilot case study: problem, architecture, decisions, results. Publish to GitHub.	**Case study published**
7	**Final Sign-off**	Complete PRD gap analysis. Version tagging.	Final PRD requirements check. Tag v1.1-release. Archive build plan. System handed to production.	**JOBPILOT v1.1 HARDENED + LIVE**

Section 4: SDE Interview Competence Map

Each component you build maps directly to SDE interview topics.

Interview Domain	What You Built	Interview Application
System Design	Multi-agent orchestration via LangGraph with state persistence, HITL checkpoints, cascading fallback	"Design a job matching system" — you built one. Agent decomposition, state machines, HITL, scaling.
API Design	CascadingLLM with retry, Lever/Greenhouse/Apollo integrations, MCP server tools	"Design an API gateway with fallback" — circuit breakers, timeouts, idempotency.
Data Modelling	30-column tracker, pgvector embeddings, feedback store, version metadata	"Design a schema for X" — normalisation, indexes, JSON vs relational trade-offs.
Distributed Systems	Concurrent agents, APScheduler coordination, Docker multi-service	"Handle concurrent writes?" — transaction isolation, locking, idempotent operations.
ML/AI Integration	RAG pipeline, LLM-as-judge, multi-agent tournament consensus	"Integrate LLMs into a product?" — prompt eng, embeddings, cost optimisation, eval loops.
Monitoring	15+ Prometheus metrics, 5 Grafana dashboards, 4 alert rules, LangSmith	"Monitor production services?" — concrete metrics, thresholds, dashboards you built.
Security	OAuth 2.0 (Gmail), no-raw-email-storage, token encryption	"Handle credentials?" — OAuth flow, refresh tokens, encrypted storage, least privilege.
Testing	Unit per agent, integration pairs, load test (150 concurrent), 24hr soak	"Test a complex system?" — test pyramid, soak testing, LLM output validation.

Section 5: Cumulative Hours Breakdown

Week	Focus	Learn (hrs)	Build (hrs)	Total
1	LLM Fundamentals + Scaffolding	9	13	**22**
2	Scout Agent	6	16	**22**
3	RAG + Resume Agent	8	14	**22**
4	LangGraph + Tracker + Application	7	15	**22**
5	Gmail + Outreach Agent	5	17	**22**
6	Email Intelligence Agent	4	18	**22**
7	Pipeline Hardening + MVP Ship	3	19	**22**

Week	Focus	Learn (hrs)	Build (hrs)	Total
	STAGE 1 SUBTOTAL	42	112	154
8	Adaptive Resume Engine	6	16	
9	Follow-up Agent + MCP Servers	5	17	
10	Prometheus + Grafana	4	18	
11	Final Deployment + Testing	3	19	
	STAGE 2 SUBTOTAL	18	70	88
12	Hardening + Portfolio	2	20	
	STAGE 3 SUBTOTAL	2	20	22
	GRAND TOTAL	62	202	264

Learning constitutes 23.5% of total time (62 of 264 hours). Front-loaded: Weeks 1–3 spend ~40–45% on learning; Weeks 8–12 spend under 15%. The expanded weekly hours primarily augment the Build column — deeper testing, more real-world data, and buffer for hard days.

Section 6: Recommended Daily Routine

At 22 hrs/week across 7 days, the average session is 3.1 hours. A consistent block outperforms scattered fragments. Two viable structures:

Structure	Weekday	Weekend
Option A: Uniform	3 hrs/day, 7 days	3 hrs/day, 7 days
Option B: Weighted weekend	2.5 hrs/day, Mon–Fri	4.75 hrs/day, Sat–Sun
Option C: 6-day	3.5 hrs/day, Mon–Sat	Rest day Sunday

Block	Duration	Activity
Orientation	5 min	Open this document. Read today's row. Identify Learn and Build tasks.

Block	Duration	Activity
Learn Block	30–60 min	Read docs, watch the video, or study the code pattern. Scale to concept difficulty: trivial (Docker) = 15 min; novel (LangGraph, RAG) = 60 min.
Build Block	Balance of session	Write the code in the Build column. Test it. Commit to git.
Close	5 min	git commit with week/day reference (e.g., W3D4: LaTeX parser). Note blockers in a running BLOCKERS.md.

Section 7: Plan-Specific Risks

Risk	Prob.	Mitigation
LangGraph steeper than estimated	High	Week 4 has 3 dedicated days plus a full buffer Day 7. If struggling by Day 3: simplify to plain Python orchestrator for MVP, migrate to LangGraph in Stage 2. The extra hours in v1.1 make this fallback painless.
LinkedIn scraping triggers restriction	Medium	Apollo.io is primary. LinkedIn is fallback at 30/day. Use throwaway account for dev. Never automate InMail.
Week is under 20 hrs due to external obligations	Medium	At 20 hrs the schedule remains viable but Week 12 hardening compresses. Under 18 hrs in any single week: defer one optional deliverable to next week. The 54–94 hr buffer above v1.0 absorbs 2–3 such weeks.
Free-tier LLM quality too low for rewriting	Medium	CascadingLLM routes to Claude Sonnet for quality-critical tasks. Zero-budget: accept ~15–20% lower phrasing sophistication.
Email misclassifies interview as rejection	Low	0.85 confidence threshold routes ambiguous to Telegram. No auto-action below threshold.
ARE degrades resume quality	Medium	Performance floor: <10% acceptance for 3 cycles → auto-revert. /pin freezes known-good version.
Motivation drops at Week 10	Medium	MVP ships at Week 7 and is immediately useful. Weeks 8–12 enhance a system you use daily. Week 12 portfolio work has direct job-search ROI.

Section 8: Changelog — v1.0 → v1.1

Change	Detail
Weekly hours	14 hrs/week → 22 hrs/week (range: 20–24 hrs)
Total hours	168 hrs → 264 hrs (+96 hrs, +57%)
MVP milestone	End of Week 8 → End of Week 7 (one week earlier)

Change	Detail
Full system milestone	End of Week 12 → End of Week 11 (one week earlier)
New Stage 3	Week 12 repurposed as dedicated hardening + portfolio week
Daily session length	2.0 hrs → ~3.1 hrs average
Per-week build hours	+8 hrs/week on average, concentrated in Build column
Test coverage	Basic per-agent → >90% coverage + 3x load test + 24hr soak
Load test scale	50 concurrent postings → 150 concurrent postings
Portfolio work	Not in scope → Week 12 Day 6: published case study
Learning hours	43 hrs → 62 hrs (+44%); ratio maintained at ~23–25% of total

Document generated: 01 April 2026 – 23:00 IST